Home Lab Project Week 10, Day 3

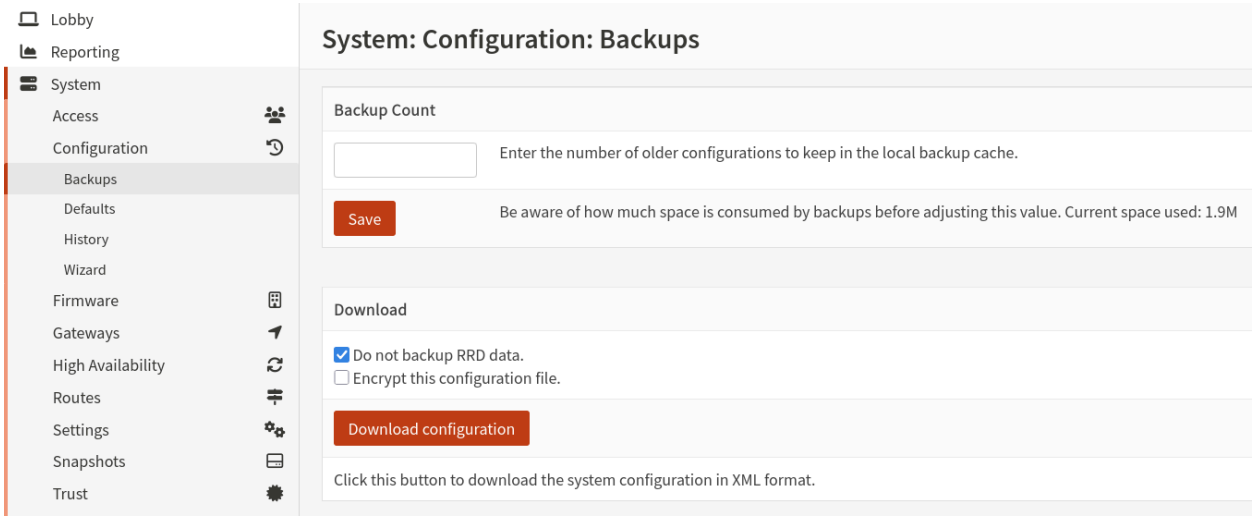
Task:

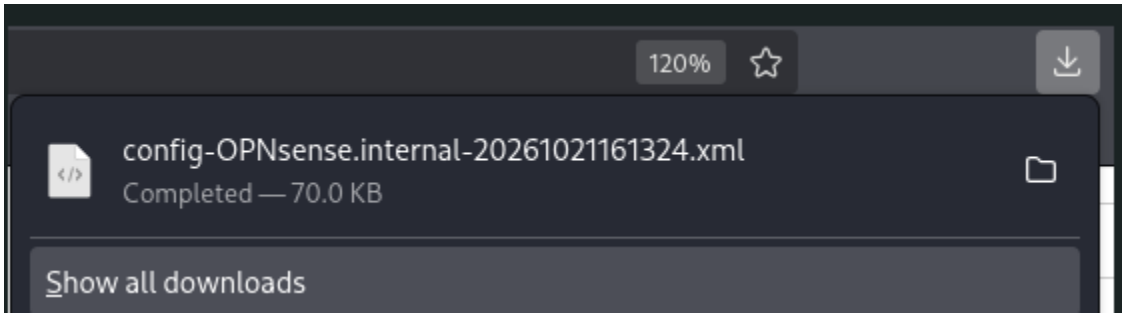
Configure firewall rules controlling traffic between the DMZ, management LAN, and other segments, restricting traffic initiated by the DMZ system toward the management network.

Week 10 Day 3 flips the direction. Days 1–2 controlled traffic **into** the DMZ. Today controls traffic the DMZ server **starts on its own**. This matters because Ubuntu is the vulnerable machine: if an attacker compromises it, their next move is to pivot into the management LAN or take over the firewall.

The goal behind this task is that management can reach into the DMZ, but the DMZ can never reach back into management.

Step 0: Check what already exists

1. Go to **Firewall** → **Rules** → **DMZ**. You may have built Ubuntu → Kali isolation rules in an earlier session. If so, today is review, adjust to the table below, and collect evidence.
2. Back up first: **System** → **Configuration** → **Backups** → **Download**.
3. 



Of course it is in Kali downloads.

What a new OPNsense interface does by default: a freshly added interface like DMZ has **no pass rules**, so OPNsense already blocks everything Ubuntu starts. You still write explicit, **logged** block rules today. Without them you have no evidence and no documented policy, just an accident of defaults.

Step 1: Create a “private networks” alias

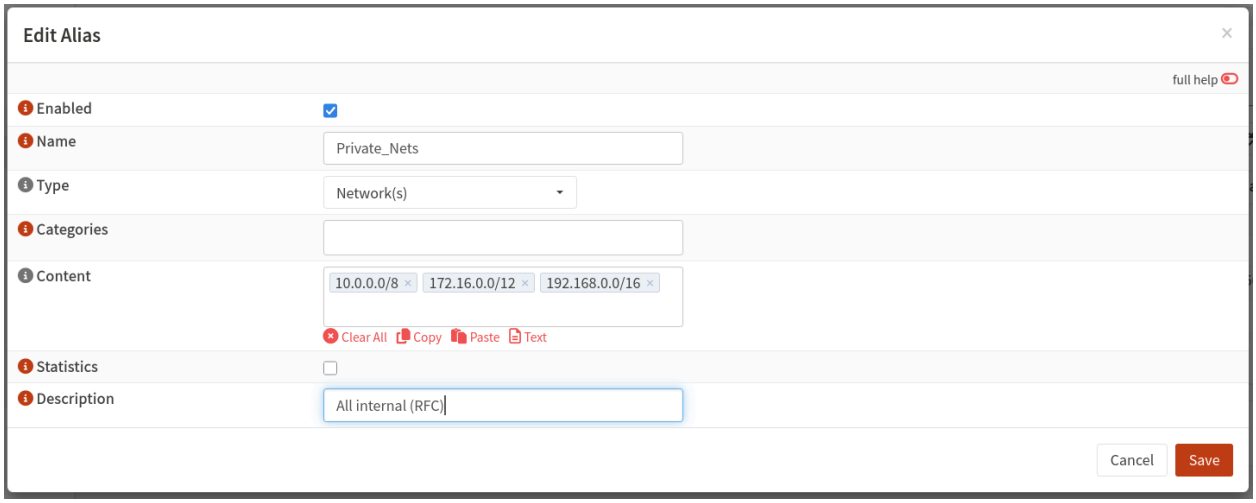
Part A: Create the `Private_Nets` alias

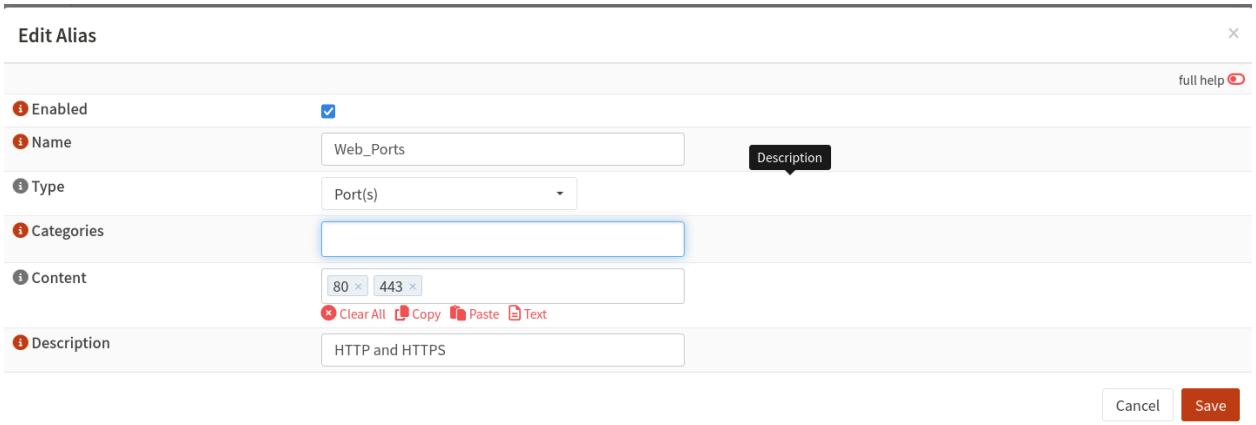
1. Log in to the OPNsense web GUI from Kali at `https://192.168.10.1`.
2. Click **Firewall** → **Aliases**.
3. Click the **+** button to add an alias.
4. Make sure **Enabled** is checked.
5. In **Name**, type `Private_Nets`.
6. In **Type**, select **Network(s)**.
7. In **Content**, type `10.0.0.0/8` and press **Enter**.
8. Type `172.16.0.0/12` and press **Enter**.
9. Type `192.168.0.0/16` and press **Enter**.
10. In **Description**, type `All internal (RFC 1918) address ranges`.

    This is used it to say “the internet, but **not** any internal network.”

11. Click **Save**.
12. Click **Apply**.

You’ll use it to say “the internet, but **not** any internal network.”

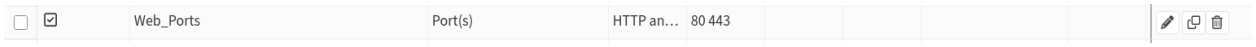




Part B: Check the `Web_Ports` alias

13. Still on Firewall → Aliases, find `Web_Ports` in the list.
14. Click the pencil icon to open it.
15. Confirm Type is Port(s) and Content shows `80` and `443`.
16. If either port is missing, type it into Content and press Enter.
17. Click Save, then Apply.

(If `Web_Ports` doesn't exist, click +, name it `Web_Ports`, set Type to Port(s), add `80` and `443`, then Save and Apply.)



Part C: Rule 1 — Allow DMZ DNS to firewall

18. Click Firewall → Rules → DMZ.
19. Click + to add a rule.
20. Action: Pass
21. Interface: DMZ
22. Protocol: TCP/UDP
23. Source: DMZ net
24. Destination: DMZ address
25. Destination port range: set both From and To to DNS (53).
26. Log: leave unchecked.
27. Description: `Allow DMZ DNS to firewall`
28. Click Save. Don't click Apply yet.

Part D: Rule 2 — Block DMZ to management LAN

29. Click + to add a rule.
30. Action: Block
31. Protocol: any
32. Source: DMZ net
33. Destination: LAN net
34. Log: check the box.
35. Description: `Block DMZ to management LAN`
36. Click Save.

Part E: Rule 3 — Block DMZ to firewall admin

37. Click + to add a rule.
38. Action: Block
39. Protocol: any
40. Source: DMZ net
41. Destination: This Firewall
42. Log: check the box.
43. Description: `Block DMZ to firewall admin`
44. Click Save.

(If This Firewall isn't in the dropdown, choose DMZ address instead, save, then repeat steps 37–44 with LAN address as the destination.)

Part F: Rule 4 — Allow DMZ web out (updates only)

45. Click + to add a rule.
46. Action: Pass
47. Protocol: TCP
48. Source: DMZ net
49. Check the Invert box next to Destination.
50. Destination: Private_Nets
51. Destination port range: set both From and To to Web_Ports.
52. Log: leave unchecked.
53. Description: `Allow DMZ web out (updates only)`
54. Click Save.

Part G: Rule 5 — Default deny DMZ outbound

55. **Click + to add a rule.**
56. **Action: Block**
57. **Protocol: any**
58. **Source: DMZ net**
59. **Destination: any**
60. **Log: check the box.**
61. **Description: `Default deny DMZ outbound`**
62. **Click Save.**

Part H: Order, apply, and save evidence

63. **Look at the DMZ rules list and confirm the order from top to bottom is: DNS pass, Block to LAN, Block to firewall, Web out pass, Default deny.**
64. **If any rule is out of place, select it and use the move arrows to fix the order.**
65. **Click Apply changes.**
66. **Take a screenshot of the full DMZ rules list.**
67. **Take a screenshot of the `Private_Nets` and `Web_Ports` aliases.**

Part I: Quick check from Ubuntu

68. **On Ubuntu, run `nslookup google.com 192.168.20.1`. It should return an answer.**
69. **Run `ping -c 2 192.168.10.10`. It should time out.**
70. **Screenshot both results.**

If both behave as expected, the rules are working and you're ready for the full tests (D1–D6 from Ubuntu, M1–M2 from Kali).

Step 2: Create the rules on the DMZ interface

These go on **DMZ** because that is where Ubuntu's traffic *enters* the firewall.

Order	Action	Protocol	Source	Destination	Port	Log	Description
1	Pass	TCP/UDP	DMZ net	DMZ address	53	Off	Allow DMZ DNS to firewall
2	Block	any	DMZ net	**LAN net**	any	**On**	Block DMZ → management LAN
3	Block	any	DMZ net	**This Firewall**	any	**On**	Block DMZ → firewall admin
4	Pass	TCP	DMZ net	**!** Private_Nets	80, 443	Off	Allow DMZ web out (updates only)
5	Block	any	DMZ net	any	any	**On**	Default deny DMZ outbound

What each rule does:

- **Rule 1** lets Ubuntu resolve names through the firewall. It sits above rule 3 because DNS lives on the firewall, and rule 3 would otherwise block it.
- **Rule 2** is the core requirement of the task: no new connections from the DMZ to the management LAN.
- **Rule 3** blocks the GUI (443) and SSH (22) on **every** firewall address, including `192.168.20.1`. The anti-lockout rule only protects the LAN, so the DMZ gets no automatic access, and this rule makes that explicit.
- **Rule 4** handles "other segments," meaning the internet. Ubuntu can still run `apt update`, but the `!` (invert) on `Private_Nets` means this rule can never be used to reach an internal network. To set it up, tick **Destination / Invert** and pick `Private_Nets`. Skip this rule if your instructor wants the DMZ fully offline.
- **Rule 5** catches everything else and logs it.

Why replies to Kali still work: the firewall is stateful. When Kali starts an SSH session, the replies from Ubuntu belong to *Kali's* connection, so OPNsense lets them back even though rule 2 exists. Rule 2 only stops connections that **Ubuntu** starts.

Drag the rules into the order shown, then **Apply changes**.

Step 3: Test from Ubuntu (blocked direction)

ID	Test	Command	Expected
D1	Ping Kali	`ping -c 4 192.168.10.10`	Times out

D2	SSH to Kali	`nc -zv -w 3 192.168.10.10 22`	Times out
D3	GUI on LAN side	`curl -k -m 5 https://192.168.10.1`	Times out
D4	GUI on DMZ side	`curl -k -m 5 https://192.168.20.1`	Times out
D5	DNS via firewall	`nslookup google.com 192.168.20.1`	Answers
D6	Updates	`sudo apt update`	Works (if rule 4 is used)

If `nc` isn't installed on Ubuntu, run `sudo apt install netcat-openbsd` first. Rule 4 has to be in place for that to work.

Step 4: Confirm it's one-way (test from Kali)

ID	Test	Command	Expected
M1	SSH to Ubuntu	`nc -zv -w 3 192.168.20.10 22`	Succeeded
M2	Ping Ubuntu	`ping -c 4 192.168.20.10`	Replies

D1–D4 failing while M1–M2 succeed shows the requirement is met: management reaches in, and the DMZ can't reach out.

Evidence

- `Private_Nets` alias
- DMZ rules list, showing the order
- Terminal output for D1–D6 and M1–M2
- **Firewall** → **Log Files** → **Live View** filtered to `192.168.20.10`, showing red block entries from rules 2, 3, and 5 during D1–D4

Day 4 (testing segmentation with scans) builds on this with nmap from both sides. Want me to create the Week 10 Day 3 lab report doc with these tables and Pass/Fail dropdowns?